

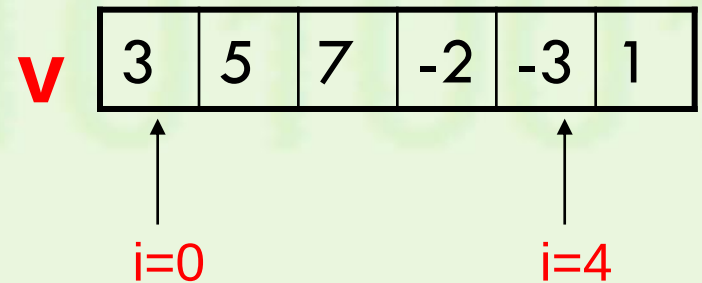
Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 12

Vettore / array

- Un array è un **tipo di dato strutturato** che aggrega un insieme di variabili dello stesso tipo.
- L'accesso ad un determinato elemento dell'array si effettua mediante un **indice** che ne rappresenta il numero d'ordine ed è di tipo int.
- La **dimensione dell'array** è il numero dei suoi elementi.
- Come ogni variabile, affinché il compilatore possa allocare lo spazio di memoria necessario, **l'array deve essere dichiarato**.



V[0]=3
V[1]=5
V[2]=7
V[3]=-2

Vettori (Array)

- Un vettore o *array* è costituito da ***elementi*** in cui è possibile memorizzare valori di tipo omogeneo, detto ***tipo dell'array***.
- Ogni elemento è individuato da un numero progressivo, detto ***indice***.
- Il primo indice è **0**. L'ultimo indice è uguale al ***numero totale di elementi - 1***.
- Il numero complessivo degli elementi del vettore viene detto ***lunghezza***.

Dichiarazione di un array

- ***tipo*** *nome_array* [***dimensione***];

Dove *tipo* specifica il tipo di ogni elemento dell'array, mentre *dimensione* definisce il numero di elementi contenuti nell'array.

Nel seguente esempio viene dichiarato un array, con nome *v*, di 6 elementi di tipo intero:

```
int v [6];
```

<i>v</i>	3	5	7	-2	-3	1
----------	---	---	---	----	----	---

Allocazione di un array

- Al momento della dichiarazione vengono allocate tante celle di memoria **consecutive** corrispondenti agli elementi dell'array. A ciascuna di essa si può accedere tramite il **nome del vettore e l'indice**.
- Ad un array **non** si può accedere nella sua interezza perché, non è una variabile semplice, ma posso accedere solo ai suoi elementi.

Accesso agli elementi di un array

- Data la dichiarazione dell'array

```
int v [6];
```

- Per accedere ad un elemento dell'array occorre specificarne nome seguito da l'indice tra parentesi quadre, ad esempio: **v[4]** , **v[2]**
- Il primo elemento di un array ha indice 0. Pertanto, la dichiarazione dell'array precedente implica la creazione di 6 elementi che vanno da v[0] a v[5].

v	3	5	7	-2	-3	1
----------	---	---	---	----	----	---

ATTENZIONE!

- Il compilatore C non restituisce alcun errore se viene superato il limite di un array; è affidata al programmatore la verifica dell'indice per evitare che questo superi i limiti dimensionali.

```
printf("%d", v[6]);
```

non darà alcun errore in compilazione ma verrà visualizzato a video il contenuto della locazione di memoria adiacente a v[5].

v	3	5	7	-2	-3	1
----------	---	---	---	----	----	---

Memoria occupata da un array

- La quantità di memoria richiesta da un array monodimensionale è legata alla dimensione ed al tipo:

$$\textit{byte totali} = \textit{sizeof(tipo)} * \textit{dimensione}$$

Inizializzazione di un array

Come ogni variabile, è possibile inizializzare un array al momento della dichiarazione. La forma generale dell'inizializzazione di un array è la seguente:

tipo nome_array [dim]={elenco_valori};

dove *elenco_valori* è un elenco di valori separati da virgole, in cui il primo valore sarà posizionato nella primo casella dell'array, il secondo nella seconda e così via.

ESEMPIO:

```
int a[10]={10,20,30,40,50,60,70,80,90,100};
```

Dopo la dichiarazione `a[0]` conterrà il valore 10, `a[1]` il valore 20 e così via tutti gli altri.

Inizializzazione di un array

- **ATTENZIONE:** L'inizializzazione deve essere contestuale alla dichiarazione
- Esempio:

```
int a[4];
```

```
a = {11, 22, 33, 44};
```

ERRORE

Inizializzazione di un array

- Se ci sono meno inizializzatori di elementi, quelli rimanenti vengono posti a 0

- Esempio:

```
int n[10] = {3};
```

azzerà i rimanenti 9 elementi del vettore

- Esempio:

```
float af[5] = {0.0}
```

pone i 5 elementi pari a 0.0

- Esempio:

```
int x[5] = {}; ERRORE
```

Inizializzazione di un array

- Se ci sono più inizializzatori di elementi, si ottiene un errore di sintassi
- Esempio:

```
int v[2] = {1, 2, 3}; ERRORE
```

Inizializzazione di un vettore

- Se si mette una lista di inizializzatori, si può evitare di specificare la lunghezza perché viene presa la lunghezza della lista.

- Esempio:

`int n[] = {1,2,3};` equivale a

`int n[3] = {1,2,3};`

Inizializzazione di un array

- E' possibile anche inizializzare un vettore all'interno del programma utilizzando un ciclo.
- Esempio:

```
int i;  
float v[4];  
for (i=0; i<4; i++) {  
    v[i]=0.0;  
}
```

Immettere un vettore da tastiera

- Anche l'immissione dei vari elementi del vettore avviene attraverso un ciclo sull'indice.
- Se voglio immettere un array 'v' non posso utilizzare lo scanf in questo modo:

`scanf ("%d", &v) ;` **NO!**

- ma è utile utilizzare il costrutto **for**
- Esempio:

```
float v[4] ;  
for (i=0; i<4; i++) {  
    scanf ("%f", &v[i]) ;  
}
```

Scorrere e visualizzare un vettore

- Come detto prima, l'accesso agli elementi di un array avviene attraverso indice.
- Se voglio stampare un array 'v' non posso utilizzare la printf in questo modo:

```
printf("%d", v);
```

NO!

- Se voglio scorrere/stampare un intero array è utile utilizzare il costrutto for
- Esempio:

```
float v[4];  
for (i=0; i<4; i++) {  
    printf("%f", v[i]);  
}
```

Operatori sugli array

- Non esistono operatori specifici per i vettori. Si può solo operare singolarmente sugli elementi che lo compongono.
- Non è possibile l'assegnamento diretto tra vettori

```
int v[10], w[10];
```

...

```
v = w; /* non è la copia del vettore w in v */
```

```
v+w; /* non è la somma dei vettori v e w */
```

Ecc...

Usare i vettori nelle funzioni

- Se il vettore è unidimensionale la dimensione può non essere specificata.

```
int funzione (int a[ ] )  
{...  
}
```

- Nel main, il parametro attuale è dato senza parentesi:

```
int n, b[10];  
.....  
n= funzione (b);
```

ATTENZIONE! I vettori sono passati per riferimento. Vengono modificati dalle funzioni a cui sono passati.

Vediamo degli esempi molto
particolari

```
/* lancio un dado 6000 volte versione vecchia*/
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
main()
{
    int faccia, tiro, freq1 = 0, freq2 = 0,
        freq3 = 0, freq4 = 0,
        freq5 = 0, freq6 = 0;

    srand(time(NULL));

    for (tiro = 1; tiro <= 6000; tiro++) {
        faccia = 1 + (rand() % 6);
        switch (faccia) {
            case 1:
                ++freq1;
                break;
            case 2:
                ++freq2;
                break;
```

```
        case 3:
            ++freq3;
            break;
        case 4:
            ++freq4;
            break;
        case 5:
            ++freq5;
            break;
        case 6:
            ++freq6;
            break;
    }
}

printf("%s%13s\n", "Faccia", "Frequenza");
printf("    1%13d\n", freq1);
printf("    2%13d\n", freq2);
printf("    3%13d\n", freq3);
printf("    4%13d\n", freq4);
printf("    5%13d\n", freq5);
printf("    6%13d\n", freq6);
return 0;
}
```

```
/* Tiro un dado 6000 volte - versione con array*/
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 7

main() {
    int f, tiro, freq[SIZE] = {0};

    srand(time(NULL));

    for (tiro = 1; tiro <= 6000; tiro++) {
        f = rand() % 6 + 1;
        freq[f]++;
    }

    printf("%s%17s\n", "Faccia", "Frequenza");

    for (f = 1; f <= SIZE - 1; f++)
        printf("%4d%17d\n", f, freq[f]);
    return 0;
}
```

Il crivello di Eratostene

(calcolo di tutti i numeri primi $< n$)

- Uso un array booleano C di dimensione n
 - $C[i] = 1$ significa che i è primo
 - $C[i] = 0$ significa che i NON è primo
- All'inizio $C[i] = 1$ per ogni i
- Realizzo $n-1$ cicli che iniziano ogni volta dall'elemento che contiene il primo 1
- Se un ciclo comincia da $c[i]$ allora assegno 0 a tutti i $c[j]$ tale che j è un multiplo di i

The end